

DSA STUDY BLUEPRINT SERIES

52. Middle of a Linked List

Finding Middle Nodes using Slow & Fast pointers

Section 07 • C++ Core Data Structures & Algorithms

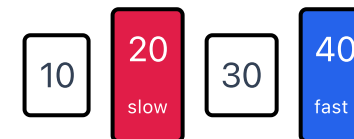
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Tortoise & Hare:** Move **slow** pointer by 1 step and **fast** pointer by 2 steps.
- When fast reaches tail, slow pointer points to the middle node.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
Node* middleNode(Node* head) {  
    Node *slow = head, *fast = head;  
    while (fast && fast->next) {  
        slow = slow->next;  
        fast = fast->next->next;  
    }  
    return slow;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(1)$

Key Practices & Gotchas

- **Important:** Returns the second middle node in lists with an even number of elements.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace the re-wirings of three node pointers reversing segment 10 → 20:

Step	Pointers [prev, curr, next]	Pointer reassignment	Resulting node state
1	[nullptr, 10, 20]	curr->next = prev	10 points to nullptr
2	[10, 20, nullptr]	curr->next = prev	20 points to 10 (Reversed segment completed!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pointer Re-linking:** Modifying node transitions requires dummy nodes to isolate head pointer alterations.
- **Fast & Slow Pointers:** Useful for loop check detections, middle-node finding, and cyclic lists splits.
- **Related LeetCode Problems:** #206 Reverse Linked List, #141 Linked List Cycle, #23 Merge k Sorted Lists.